

网络安全创新创业实践

第五次作业

基于 TenSEAL CKKS 的
单输入单输出密文卷积

姓名：_____

学号：_____

班级：_____

完成日期：2026 年 7 月 24 日

课程实验报告

摘要

本作业选择开源全同态加密库 TenSEAL 0.3.16，以 Microsoft SEAL 为后端，完成单输入、单输出的 4×4 输入与 3×3 卷积核密文卷积。步长为 1、填充为 0，按 CNN Conv2D 的互相关语义得到 2×2 输出。客户端使用 CKKS 参数 $N = 8192$ 、系数模数位长链 $[60, 40, 40, 60]$ 和全局缩放因子 2^{40} ，通过 `im2col_encoding` 将四个滑动窗口打包并加密；服务器只持有不含私钥的公开评估上下文，使用 `conv2d_im2col` 完成密文与明文卷积核的乘加，客户端最终解密。

明文手算和独立 Python 参考实现均得到 $[\frac{26}{46} \frac{31}{51}]$ 。对完整流程执行 10 次独立试验，10 个输入密文的 SHA-256 均不同，全部输出满足 10^{-3} 绝对误差阈值；所有试验的最大绝对误差为 6.882×10^{-6} ，平均绝对误差为 5.106×10^{-6} 。服务器上下文的 `has_secret_key` 和 `is_private` 均为 `false`。中位耗时分别为加密与序列化 10.402ms、服务器加载/计算/序列化 14.575ms、客户端加载与解密 2.328ms。报告给出原始要求、资料学习、算法推导、威胁模型、代码、运行结果、可视化、自动化测试、局限和参考文献。

关键词：全同态加密；CKKS；TenSEAL；Microsoft SEAL；密文卷积；im2col；SIMD 打包

目录

1	作业原文、任务解释与验收标准	1
1.1	作业原文	1
1.2	严格化解释	1
2	参考资料学习与技术选型	1
2.1	课程资料要点	1
2.2	开源库比较	2
3	理论基础与安全边界	2
3.1	RLWE 与 CKKS 近似语义	2
3.2	同态线性运算	3
3.3	威胁模型	3
4	卷积定义、手算与 im2col	4
4.1	CNN Conv2D 语义	4
4.2	四个输出的独立手算	4
4.3	im2col 变换	5
5	程序设计与实现	5
5.1	代码结构	5

5.2 客户端/服务器分离的核心代码	6
6 参数选择与正确性判据	7
6.1 CKKS 参数	7
6.2 误差判据	7
7 运行结果与验证	8
7.1 环境与控制台结果	8
7.2 数值误差	9
7.3 时间与空间开销	10
8 自动化测试与复现方法	11
8.1 测试结果	11
8.2 完整复现命令	11
9 讨论、局限与改进方向	11
9.1 为什么本题不需要 Bootstrapping	11
9.2 实现局限	11
9.3 可扩展方向	12
10 结论	12
A 交付文件索引	13
B 完整源代码	13
B.1 fhe_convolution.py	13
B.2 test_fhe_convolution.py	19
B.3 visualize_results.py	21

1 作业原文、任务解释与验收标准

1.1 作业原文

“第五次作业”目录中 实验要求.txt 的原文为：

作业 5：选择任一开源全同态加密库（CPU 或 GPU 均可），使用单输入单输出 4×4 输入和 3×3 卷积核（步长 1，无填充），实现密文卷积并验证结果正确性。

1.2 严格化解释

本报告将要求转换为以下可验证条件：

1. 开源库必须是真正实现基于格的同态加密，而非自行模拟“加密后计算”；本实验选用 Apache-2.0 许可的 TenSEAL，并固定版本 0.3.16。
2. “单输入单输出”解释为卷积层输入通道数 $C_{in} = 1$ 、输出通道数 $C_{out} = 1$ 。张量形状为 $[1, 1, 4, 4]$ ，卷积核形状为 $[1, 1, 3, 3]$ 。
3. 步长 $S = 1$ 、填充 $P = 0$ ，故输出高宽必须均为

$$\left\lfloor \frac{4 + 2P - 3}{S} \right\rfloor + 1 = 2.$$

4. 输入必须在服务器计算期间保持密文态；服务器上下文不得含私钥。卷积核是服务器明文模型参数，本实验保护输入隐私，不声称保护模型参数。
5. 正确性由手工推导、独立明文实现、密文解密结果三方交叉验证。CKKS 是近似方案，因此以预先固定的绝对误差阈值 10^{-3} 判断，而非要求浮点逐位相等。
6. 保留参数、密钥边界、密文大小、计时、每次试验误差、密文哈希、测试输出和完整代码，保证结果可复核。

2 参考资料学习与技术选型

2.1 课程资料要点

目录中的《全同态密码算法和应用》从 LWE/RLWE 出发，比较 BFV、BGV、CKKS 和 TFHE，并指出卷积属于线性乘加：可使用 SIMD 打包、Rotation 和明文权重乘法实现。其 4×4 输入、 3×3 核、步长 1、无填充的示例与本题完全对应，还给出四个有效输出槽和 9 个卷积偏移。资料同时强调 CKKS 的 scale、level、Rescale、KeySwitch 和 Galois Key，以及 TenSEAL 适合快速验证加密张量计算 [7]。

《对称密码的软件实现与优化》和《SM3 软件实现》主要讨论 CPU 寄存器、SIMD、指令级并行、缓存、内存访问和不同架构的性能差异 [8, 9]。它们并不直接定义 FHE 卷

积，但提供了两条重要工程原则：第一，性能结论必须标注硬件和软件环境；第二，不能只统计抽象运算次数，还应关注大密钥和密文造成的内存流量。本实验因此同时记录计算时间和序列化大小。

2.2 开源库比较

表 1: 候选开源库与本题适配性

库	主要方案	与本题相关特点	选择结论
Microsoft SEAL	BFV、BGV、CKKS	成熟 C++ 后端, 参数检查严格; 需要自行完成更多张量封装	可行, 开发量较大
OpenFHE	BGV、BFV、CKKS、FHEW、TFHE	方案全面, 适合深入研究和复杂评测	可行, 接口较重
TenSEAL	BFV、CKKS	基于 SEAL, 直接提供加密向量、 <code>im2col</code> 和卷积 API[2, 3]	采用
Concrete ML	TFHE	自动量化与模型转换能力强	本题过小, 不需完整编译链

CKKS 原论文将加密噪声解释为近似计算误差的一部分，并用 Rescale 管理数值尺度，支持复数/实数向量的批处理 [1]。本题卷积只含密文与明文权重的线性乘加，乘法深度浅，无需 Bootstrapping；这与 CKKS 和 TenSEAL 的优势吻合。

3 理论基础与安全边界

3.1 RLWE 与 CKKS 近似语义

CKKS 建立在多项式环

$$R_q = \mathbb{Z}_q[X]/(X^N + 1)$$

上的 RLWE 安全结构之上。编码器把槽向量 $\mathbf{z} \in \mathbb{C}^{N/2}$ 通过典范嵌入映射到环元素，并乘缩放因子 Δ 后取整。简化地写，私钥 s 下的二元密文满足

$$c_0 + c_1 s = m + e \pmod{q},$$

其中 m 是编码后的消息, e 是小噪声。解密和解码恢复的是近似值。与 BFV/BGV 的精确整数语义不同, CKKS 的正确性应表述为

$$\|\text{Decode}(\text{Dec}(ct)) - z\|_{\infty} \leq \varepsilon.$$

本实验在运行前固定 $\varepsilon = 10^{-3}$, 避免观察结果后再放宽阈值。

3.2 同态线性运算

卷积核由服务器持有, 因而每个权重是明文常量。密文加法和密文乘明文分别为

$$(c_0, c_1) + (d_0, d_1) = (c_0 + d_0, c_1 + d_1),$$

$$p(c_0, c_1) = (pc_0, pc_1).$$

密文乘明文不会产生 s^2 项, 故不需要因该乘法执行重线性化; 但 SIMD 槽旋转会改变对应的密钥表示, 需要 Galois Key 完成 KeySwitch。TenSEAL 的卷积接口封装了旋转、逐槽明文乘法和累加。

3.3 威胁模型

本实验采用诚实但好奇的服务器模型: 服务器按协议计算, 但可能试图从收到的数据推断用户输入。客户端持有私钥、生成 Galois Key、加密输入并解密结果; 服务器只得到公钥、评估密钥、加密输入和明文卷积核。评估密钥允许特定同态变换, 但不等价于解密密钥。程序在服务器函数入口检查:

```
is_private=False,    has_secret_key=False.
```

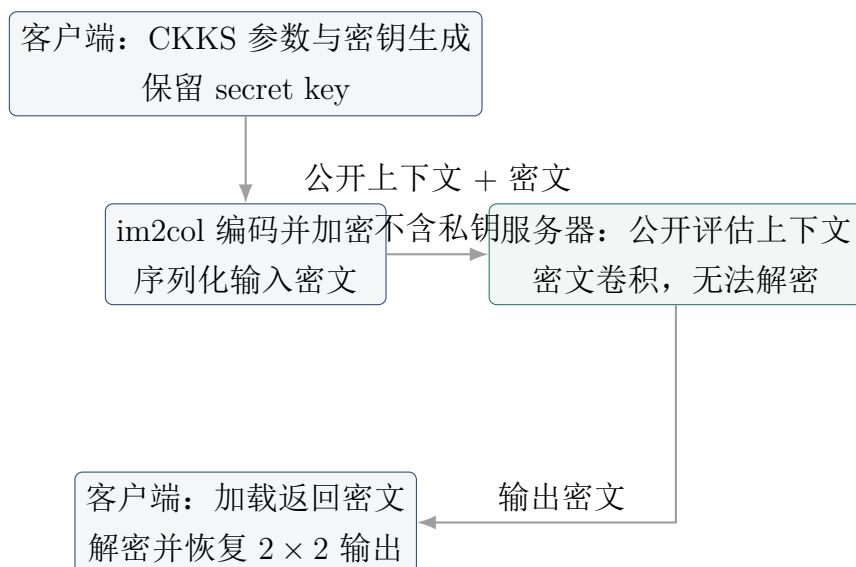


图 1: 客户端与服务器的密钥和数据边界

边界说明

本实验只保护客户端输入。服务器知道卷积核、参数、输入形状和输出形状，也能观察密文长度与计算时序；未实现模型隐私、访问模式隐藏、主动恶意服务器证明或侧信道防护。实验使用库提供的密码实现，不把自行编写的明文参考函数当作密码组件。

4 卷积定义、手算与 im2col

4.1 CNN Conv2D 语义

深度学习框架通常把名为 convolution 的算子实现为不翻转卷积核的二维互相关 [6]。对于本题，输出为

$$y_{i,j} = \sum_{u=0}^2 \sum_{v=0}^2 x_{i+u,j+v} k_{u,v}, \quad i, j \in \{0, 1\}.$$

若使用数学信号处理中“先翻转卷积核”的定义，数值会不同。因此程序和报告均显式声明采用 CNN Conv2D 语义。

实验输入与卷积核为

$$X = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & -1 & 2 \\ 0 & 3 & -2 \\ 2 & 1 & -1 \end{bmatrix}.$$

4.2 四个输出的独立手算

$$y_{0,0} = 1 - 2 + 6 + 0 + 18 - 14 + 18 + 10 - 11 = 26,$$

$$y_{0,1} = 2 - 3 + 8 + 0 + 21 - 16 + 20 + 11 - 12 = 31,$$

$$y_{1,0} = 5 - 6 + 14 + 0 + 30 - 22 + 26 + 14 - 15 = 46,$$

$$y_{1,1} = 6 - 7 + 16 + 0 + 33 - 24 + 28 + 15 - 16 = 51.$$

故明文参考输出是

$$Y = \begin{bmatrix} 26 & 31 \\ 46 & 51 \end{bmatrix}.$$

4.3 im2col 变换

将四个 3×3 窗口按行展开，得到

$$W = \begin{bmatrix} 1 & 2 & 3 & 5 & 6 & 7 & 9 & 10 & 11 \\ 2 & 3 & 4 & 6 & 7 & 8 & 10 & 11 & 12 \\ 5 & 6 & 7 & 9 & 10 & 11 & 13 & 14 & 15 \\ 6 & 7 & 8 & 10 & 11 & 12 & 14 & 15 & 16 \end{bmatrix},$$

卷积核向量为

$$\mathbf{k} = [1, -1, 2, 0, 3, -2, 2, 1, -1]^T.$$

因此卷积等价于矩阵向量乘法

$$W\mathbf{k} = [26, 31, 46, 51]^T.$$

TenSEAL 的 `im2col_encoding` 将四个窗口组织成供旋转与批量内积使用的 CKKS 向量。本次返回的逻辑窗口数为 4，加密向量长度为 64；`conv2d_im2col` 一次返回四个逻辑输出槽。

5 程序设计与实现

5.1 代码结构

表 2: 作业代码与证据文件

文件	职责
<code>fhe_convolution.py</code>	参数检查、明文参考卷积、客户端/服务器上下文分离、加密、密文计算、解密、10 次试验与 JSON 证据
<code>test_fhe_convolution.py</code>	手算结果、非法形状、步长、密钥隔离、密文卷积精度和随机化加密测试
<code>visualize_results.py</code>	从最终 JSON 生成矩阵/误差图和运行验证汇总图
<code>requirements.txt</code>	固定 TenSEAL、Matplotlib 和 NumPy 版本
<code>output/experiment_results.json</code>	参数、每次密文哈希、误差、计时、大小和最终判定
<code>output/figures/</code>	报告使用的两张程序生成图片

5.2 客户端/服务器分离的核心代码

以下代码片段体现私钥不进入服务器的关键路径：

```
1 client_context = create_client_context(parameters)
2
3 evaluation_context_bytes = client_context.serialize(
4     save_public_key=True,
5     save_secret_key=False,
6     save_galois_keys=True,
7     save_relin_keys=True,
8 )
9 server_context = ts.context_from(evaluation_context_bytes)
10
11 encrypted_input, windows, _ = client_encrypt_im2col(
12     client_context, image
13 )
14 encrypted_output = server_encrypted_convolution(
15     server_context, encrypted_input, kernel, windows
16 )
17 decrypted = client_decrypt_output(
18     client_context, encrypted_output, windows
19 )
```

服务器计算函数再次做防御性检查：

```
1 if server_context.is_private() or server_context.has_secret_key():
2     raise RuntimeError(
3         "refusing to evaluate with a secret-bearing server context"
4     )
5 encrypted = ts.ckks_vector_from(server_context, encrypted_input)
6 result = encrypted.conv2d_im2col(kernel, windows)
7 return result.serialize()
```

这种结构比在同一进程对象上直接“加密、计算、解密”更能表达真实远程推理边界。尽管实验在一台计算机上运行，数据通过序列化字节在两套上下文间传递，服务器对象确实没有私钥。

6 参数选择与正确性判据

6.1 CKKS 参数

表 3: CKKS 参数及理由

参数	取值	理由
多项式模数次数 N	8192	提供 $N/2 = 4096$ 个复数槽，足以容纳本题打包；兼顾演示速度与安全参数
系数模数位长链	[60, 40, 40, 60]	总位长 200；低于 HE Standard-/SEAL 对 $N = 8192$ 、128 bit 目标安全级别常用的 218 bit 上限 [5, 4]
全局 scale	2^{40}	提供约 40 bit 小数精度，并与中间 40 bit 素数匹配
Galois Key	生成	conv2d_im2col 需要槽旋转
Relinearization Key	序列化保留	通用评估上下文保留；本题主路径为密文乘明文，不依赖密文乘密文
Bootstrapping	不使用	卷积是浅层线性计算，模数链预算充足

TenSEAL/SEAL 在构造上下文时执行参数合法性检查。这里的“128 bit”是目标经典安全级别，不是说输出有 128 bit 数值精度。参数安全性、数值精度和计算深度是不同概念，不能混为一谈。

6.2 误差判据

对第 t 次试验，定义

$$e_{t,i,j} = |\hat{y}_{t,i,j} - y_{i,j}|, \quad E_{\infty} = \max_{t,i,j} e_{t,i,j}.$$

通过条件是

$$E_{\infty} \leq 10^{-3},$$

并且输出形状为 2×2 、服务器无私钥、10 次输入密文序列化结果的 SHA-256 全部不同。密文哈希互异只能证明本次观测到加密随机化生效，不构成对随机数生成器熵质量的形式化证明。

7 运行结果与验证

7.1 环境与控制台结果

实验环境为 Windows 11、Python 3.12.9、TenSEAL 0.3.16，后端为 Microsoft SEAL。
执行：

```
python fhe_convolution.py --trials 10 \  
--output output/experiment_results.json
```

关键输出为：

```
TEN SEAL CKKS ENCRYPTED CONVOLUTION  
=====  
input shape : [1, 1, 4, 4]  
kernel shape : [1, 1, 3, 3]  
output shape : [1, 1, 2, 2]  
stride/padding: 1/0  
plain result : [[26.0, 31.0], [46.0, 51.0]]  
decrypt result: [[26.000003036778416, 31.000004199480824],  
                 [46.000006376746114, 51.000006842397156]]  
max abs error : 6.881520243e-06  
tolerance : 1.0e-03  
trials passed : 10/10  
randomized ct : 10/10 unique  
server has sk : False  
input ct bytes: 334,372  
output ct bytes: 235,338  
median encrypt: 10.402 ms  
median server : 14.575 ms  
median decrypt: 2.328 ms  
RESULT : PASS
```

```

TENSEAL CKKS ENCRYPTED CONVOLUTION
$ python fhe_convolution.py --trials 10

[OK] shapes      [1, 1, 4, 4] x [1, 1, 3, 3] -> [1, 1, 2, 2]
[OK] parameters  N=8192, Q-bits=[60, 40, 40, 60], scale=2^40
[OK] key split    server_has_secret_key=False
[OK] expected     [[26.0, 31.0], [46.0, 51.0]]
[OK] decrypted    [[26.000003, 31.000004], [46.000006, 51.000007]]
[OK] max error    6.882e-06 <= 1.0e-03
[OK] randomized   10/10 unique ciphertexts

[SIZE] eval context 35,472,727 B | input ct 334,372 B | output ct 235,338 B
[TIME] median      encrypt 10.402 ms | server 14.575 ms | decrypt 2.328 ms

RESULT: ALL CORRECTNESS AND KEY-ISOLATION CHECKS PASSED
    
```

图 2: 程序根据最终 JSON 生成的运行验证汇总图

7.2 数值误差

最后一次试验的逐槽结果见表 4。10 次试验中的全局最大误差略大于该表最大值，但仍只有 6.882×10^{-6} 。

表 4: 最后一次密文卷积的逐槽误差

i	j	明文参考值	CKKS 解密值	绝对误差
0	0	26	26.000 003 036 778 416	3.0368×10^{-6}
0	1	31	31.000 004 199 480 824	4.1995×10^{-6}
1	0	46	46.000 006 376 746 114	6.3767×10^{-6}
1	1	51	51.000 006 842 397 156	6.8424×10^{-6}

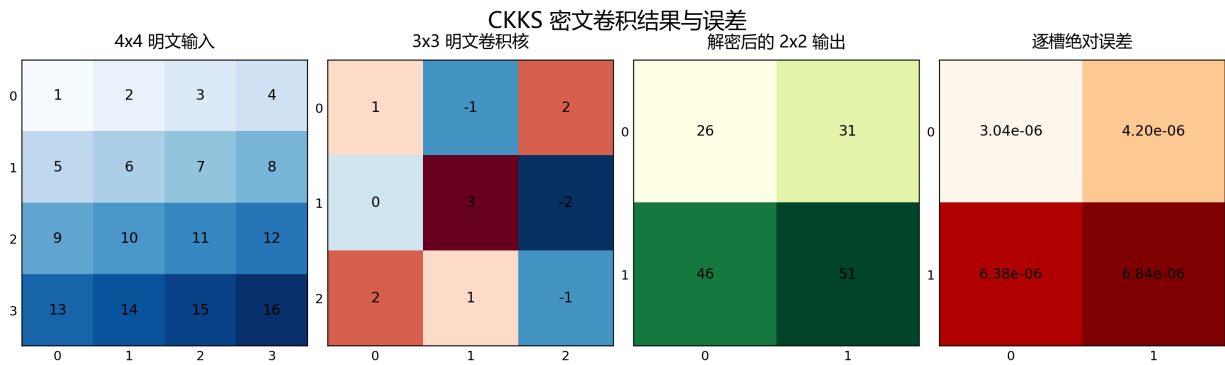


图 3: 输入、卷积核、解密输出与逐槽绝对误差

正确性结论

手算、独立明文函数和密文解密的输出形状均为 2×2 ；明文结果完全一致，10 次密文结果全部满足预先设定的 10^{-3} 误差阈值。 $E_{\infty} = 6.882 \times 10^{-6}$ ，仅为阈值的约 0.688%。因此在 CKKS 的近似语义下，结果正确。

7.3 时间与空间开销

表 5: 10 次试验的中位耗时与序列化大小

指标	数值
上下文和密钥初始化	1337.975 ms
客户端 im2col 加密与序列化中位数	10.402 ms
服务器加载、密文卷积与序列化中位数	14.575 ms
客户端加载与解密中位数	2.328 ms
客户端私有上下文序列化	705 215 B
服务器公开评估上下文序列化	35 472 727 B
输入密文序列化	334 372 B
输出密文序列化	235 338 B

原始 4×4 双精度输入仅为 128 B，而打包后输入密文约 334 KB；公开评估上下文约 35.5 MB，主要来自 Galois Keys。这说明 FHE 的主要代价不只在算术，也在密钥和密文数据移动。当前代码为通用性生成完整 Galois Key 集；生产实现可按实际旋转步选择密钥，减少传输和存储。计时仅反映本机、小规模单进程实验，不应外推为通用吞吐量结论。

8 自动化测试与复现方法

8.1 测试结果

执行 `python -m unittest -v`，结果为：

```
test_ckks_encryption_is_randomized ..... ok
test_encrypted_convolution_matches_reference ..... ok
test_plaintext_reference ..... ok
test_server_has_no_secret_key ..... ok
test_shape_and_stride_validation ..... ok
-----
```

```
Ran 5 tests in 1.611s
OK
```

测试不仅检查“程序能运行”，还覆盖以下负面条件： 4×4 之外的输入必须拒绝，步长 2 必须拒绝，服务器上下文必须不含私钥，两次相同输入加密的序列化密文必须不同。

8.2 完整复现命令

在“第五次作业”目录的 `code` 子目录中执行：

```
python -m pip install -r requirements.txt
python -m py_compile fhe_convolution.py \
    test_fhe_convolution.py visualize_results.py
python -m unittest -v
python fhe_convolution.py --trials 10 \
    --output output/experiment_results.json
python visualize_results.py
```

由于 CKKS 加密使用随机数，每次运行的密文哈希、末位近似误差和耗时会变化；明文参考矩阵、输出形状、误差阈值和通过结论应保持不变。JSON 文件保存每次试验的实际值，报告中的数字对应 2026 年 7 月 24 日的最终采样。

9 讨论、局限与改进方向

9.1 为什么本题不需要 Bootstrapping

本题只有一层线性卷积，核心是密文乘明文和密文加法。它不像深层 CNN 那样连续消耗大量 level，也没有 ReLU、MaxPool 等比较型非线性。因此使用 Leveled CKKS 的有限模数链即可完成，Bootstrapping 会增加大量不必要开销。

9.2 实现局限

1. 近似而非精确：CKKS 解密值不是逐位精确整数。若业务要求严格整数等式，可改用 BFV/BGV，并重新设计批处理和明文模数。

2. **仅保护输入**：卷积核是明文。若要求模型也保密，需要密文乘密文或多方协议，计算深度、密钥管理和开销均会变化。
3. **形状固定**：程序有意严格限定题目要求的 4×4 、 3×3 、stride 1、padding 0；这使错误输入无法悄然得到其他输出。
4. **评估密钥较大**：当前完整 Galois Key 集导致公开上下文约 35.5 MB。可根据所需 Rotation 精简密钥。
5. **性能样本有限**：仅在一台 Windows CPU 上执行 10 次，未固定 CPU 频率、线程绑定或系统负载，计时用于复现当前实验，不是严谨的跨平台基准。
6. **未覆盖恶意行为和侧信道**：服务器返回错误密文时客户端只能发现数值异常；未加入可验证计算证明，也未评估缓存、分支、功耗或通信元数据侧信道。

9.3 可扩展方向

后续可对比 BFV 精确整数卷积与 CKKS 近似卷积；针对多通道卷积研究通道/空间联合打包；只生成必要旋转的 Galois Keys；增加误差随 scale 和模数链变化的参数扫描；在相同安全级别下比较 CPU、GPU 和 FPGA 的延迟、吞吐与能耗。对于完整 CNN，还需把 BatchNorm 融合到卷积，把 ReLU/MaxPool 替换为 HE 友好近似，并在整张计算图上联合规划 level 和 scale。

10 结论

本作业使用真正的开源 FHE 库 TenSEAL 完成了规定的单输入单输出密文卷积。客户端加密 4×4 输入，服务器在无私钥上下文中对 3×3 明文卷积核执行 stride 1、padding 0 的密文卷积，客户端解密得到 2×2 输出。手算和独立明文实现均为 $[[26, 31], [46, 51]]$ ；10 次随机化加密全部通过，最大绝对误差 $6.882 \times 10^{-6} < 10^{-3}$ ，服务器私钥检查为 false。代码、JSON、图片、测试和完整报告共同形成了可运行、可验证、边界陈述准确的交付。

A 交付文件索引

相对路径	内容
code/fhe_convolution.py	密文卷积主程序和实验驱动
code/test_fhe_convolution.py	5 项自动化测试
code/visualize_results.py	两张报告图片的生成程序
code/requirements.txt	固定依赖版本
code/README.md	运行说明与安全边界
code/output/experiment_results.json	10 次试验的完整机器可读证据
code/output/figures/encrypted_convolution_result.png	矩阵与误差可视化
code/output/figures/runtime_validation.png	运行验证汇总图
report/main.tex	本报告 LaTeX 源文件
report/main.pdf	编译后的最终报告

B 完整源代码

B.1 fhe_convolution.py

```

1  """Single-channel encrypted 2-D convolution with TenSEAL CKKS.
2
3  The client owns the secret key. The server receives only a public evaluation
4  context, an encrypted im2col representation of the input, and a plaintext
5  kernel. It returns an encrypted 2 x 2 output for client-side decryption.
6  """
7
8  from __future__ import annotations
9
10 import argparse
11 from dataclasses import asdict, dataclass
12 import hashlib
13 import json
14 import math
15 from pathlib import Path
16 import platform
17 import statistics
18 import sys
19 import time
20 from typing import Sequence
21
22 import tenseal as ts

```

```

23
24
25 Matrix = list[list[float]]
26
27 DEFAULT_INPUT: Matrix = [
28     [1.0, 2.0, 3.0, 4.0],
29     [5.0, 6.0, 7.0, 8.0],
30     [9.0, 10.0, 11.0, 12.0],
31     [13.0, 14.0, 15.0, 16.0],
32 ]
33
34 DEFAULT_KERNEL: Matrix = [
35     [1.0, -1.0, 2.0],
36     [0.0, 3.0, -2.0],
37     [2.0, 1.0, -1.0],
38 ]
39
40
41 @dataclass(frozen=True)
42 class CKKSPParameters:
43     poly_modulus_degree: int = 8192
44     coeff_mod_bit_sizes: tuple[int, ...] = (60, 40, 40, 60)
45     global_scale_bits: int = 40
46     target_security_level_bits: int = 128
47
48     @property
49     def total_coeff_modulus_bits(self) -> int:
50         return sum(self.coeff_mod_bit_sizes)
51
52
53 def _validate_matrix(matrix: Sequence[Sequence[float]], rows: int, cols: int, name: str) -> None:
54     if len(matrix) != rows or any(len(row) != cols for row in matrix):
55         raise ValueError(f"{name} must have shape {rows}x{cols}")
56     for row in matrix:
57         for value in row:
58             if isinstance(value, bool) or not isinstance(value, (int, float)):
59                 raise TypeError(f"{name} entries must be real numbers")
60             if not math.isfinite(float(value)):
61                 raise ValueError(f"{name} entries must be finite")
62
63
64 def plaintext_valid_convolution(
65     image: Sequence[Sequence[float]],
66     kernel: Sequence[Sequence[float]],
67     stride: int = 1,
68 ) -> Matrix:
69     """Return mathematical cross-correlation, as used by CNN Conv2D layers."""
70     if stride != 1:
71         raise ValueError("this assignment requires stride=1")
72     _validate_matrix(image, 4, 4, "image")
73     _validate_matrix(kernel, 3, 3, "kernel")
74
75     output: Matrix = []
76     for out_row in range(2):
77         row: list[float] = []
78         for out_col in range(2):
79             value = 0.0
80             for kernel_row in range(3):
81                 for kernel_col in range(3):
82                     value += (

```

```

83         float(image[out_row + kernel_row][out_col + kernel_col])
84         * float(kernel[kernel_row][kernel_col])
85     )
86     row.append(value)
87     output.append(row)
88     return output
89
90
91 def create_client_context(parameters: CKKSPParameters) -> ts.Context:
92     context = ts.context(
93         ts.SCHEME_TYPE.CKKS,
94         poly_modulus_degree=parameters.poly_modulus_degree,
95         coeff_mod_bit_sizes=list(parameters.coeff_mod_bit_sizes),
96     )
97     context.global_scale = 2**parameters.global_scale_bits
98     context.generate_galois_keys()
99     if not context.is_private() or not context.has_secret_key():
100         raise RuntimeError("client context unexpectedly lacks the secret key")
101     return context
102
103
104 def serialize_evaluation_context(client_context: ts.Context) -> bytes:
105     """Serialize public/evaluation material while explicitly excluding sk."""
106     return client_context.serialize(
107         save_public_key=True,
108         save_secret_key=False,
109         save_galois_keys=True,
110         save_relin_keys=True,
111     )
112
113
114 def load_server_context(serialized_context: bytes) -> ts.Context:
115     context = ts.context_from(serialized_context)
116     if context.is_private() or context.has_secret_key():
117         raise RuntimeError("server context must not contain the secret key")
118     return context
119
120
121 def client_encrypt_im2col(
122     client_context: ts.Context,
123     image: Sequence[Sequence[float]],
124 ) -> tuple[bytes, int, int]:
125     _validate_matrix(image, 4, 4, "image")
126     encrypted, windows = ts.im2col_encoding(client_context, image, 3, 3, 1)
127     if windows != 4:
128         raise RuntimeError(f"expected four valid windows, got {windows}")
129     return encrypted.serialize(), windows, encrypted.size()
130
131
132 def server_encrypted_convolution(
133     server_context: ts.Context,
134     encrypted_input: bytes,
135     kernel: Sequence[Sequence[float]],
136     windows: int,
137 ) -> bytes:
138     if server_context.is_private() or server_context.has_secret_key():
139         raise RuntimeError("refusing to evaluate with a secret-bearing server context")
140     _validate_matrix(kernel, 3, 3, "kernel")
141     encrypted = ts.ckks_vector_from(server_context, encrypted_input)
142     result = encrypted.conv2d_im2col(kernel, windows)

```

```

143     return result.serialize()
144
145
146 def client_decrypt_output(
147     client_context: ts.Context,
148     encrypted_output: bytes,
149     windows: int,
150 ) -> Matrix:
151     if not client_context.has_secret_key():
152         raise RuntimeError("client context must contain the secret key")
153     values = ts.ckks_vector_from(client_context, encrypted_output).decrypt()
154     if len(values) < windows:
155         raise RuntimeError("decrypted vector is shorter than the output window count")
156     values = [float(value) for value in values[:windows]]
157     return [values[:2], values[2:4]]
158
159
160 def _flatten(matrix: Sequence[Sequence[float]]) -> list[float]:
161     return [float(value) for row in matrix for value in row]
162
163
164 def _timing_summary(values: Sequence[float]) -> dict[str, float]:
165     return {
166         "mean_ms": statistics.fmean(values),
167         "median_ms": statistics.median(values),
168         "min_ms": min(values),
169         "max_ms": max(values),
170         "stdev_ms": statistics.stdev(values) if len(values) > 1 else 0.0,
171     }
172
173
174 def run_experiment(
175     trials: int = 10,
176     tolerance: float = 1e-3,
177     image: Sequence[Sequence[float]] = DEFAULT_INPUT,
178     kernel: Sequence[Sequence[float]] = DEFAULT_KERNEL,
179 ) -> dict[str, object]:
180     if trials < 2:
181         raise ValueError("at least two trials are required to test randomized encryption")
182     if tolerance <= 0:
183         raise ValueError("tolerance must be positive")
184     _validate_matrix(image, 4, 4, "image")
185     _validate_matrix(kernel, 3, 3, "kernel")
186
187     parameters = CKKSPParameters()
188     context_start = time.perf_counter_ns()
189     client_context = create_client_context(parameters)
190     evaluation_context_bytes = serialize_evaluation_context(client_context)
191     server_context = load_server_context(evaluation_context_bytes)
192     context_setup_ms = (time.perf_counter_ns() - context_start) / 1_000_000
193
194     secret_context_bytes = client_context.serialize(
195         save_public_key=True,
196         save_secret_key=True,
197         save_galois_keys=True,
198         save_relin_keys=True,
199     )
200     expected = plaintext_valid_convolution(image, kernel)
201     expected_flat = _flatten(expected)
202

```

```

203 trial_records: list[dict[str, object]] = []
204 encryption_times: list[float] = []
205 evaluation_times: list[float] = []
206 decryption_times: list[float] = []
207 input_hashes: list[str] = []
208 all_errors: list[float] = []
209 last_decrypted: Matrix | None = None
210 input_ciphertext_size = 0
211 output_ciphertext_size = 0
212 encoded_slot_count = 0
213
214 for trial in range(1, trials + 1):
215     start = time.perf_counter_ns()
216     encrypted_input, windows, encoded_slot_count = client_encrypt_im2col(client_context, image)
217     encryption_ms = (time.perf_counter_ns() - start) / 1_000_000
218
219     start = time.perf_counter_ns()
220     encrypted_output = server_encrypted_convolution(
221         server_context,
222         encrypted_input,
223         kernel,
224         windows,
225     )
226     evaluation_ms = (time.perf_counter_ns() - start) / 1_000_000
227
228     start = time.perf_counter_ns()
229     decrypted = client_decrypt_output(client_context, encrypted_output, windows)
230     decryption_ms = (time.perf_counter_ns() - start) / 1_000_000
231
232     decrypted_flat = _flatten(decrypted)
233     errors = [abs(actual - reference) for actual, reference in zip(decrypted_flat, expected_flat)]
234     max_error = max(errors)
235     all_errors.extend(errors)
236     last_decrypted = decrypted
237     input_ciphertext_size = len(encrypted_input)
238     output_ciphertext_size = len(encrypted_output)
239     digest = hashlib.sha256(encrypted_input).hexdigest()
240     input_hashes.append(digest)
241     encryption_times.append(encryption_ms)
242     evaluation_times.append(evaluation_ms)
243     decryption_times.append(decryption_ms)
244     trial_records.append(
245         {
246             "trial": trial,
247             "input_ciphertext_sha256": digest,
248             "decrypted_output": decrypted,
249             "max_absolute_error": max_error,
250             "within_tolerance": max_error <= tolerance,
251             "timing_ms": {
252                 "client_encrypt_and_serialize": encryption_ms,
253                 "server_load_evaluate_and_serialize": evaluation_ms,
254                 "client_load_and_decrypt": decryption_ms,
255             },
256         }
257     )
258
259 assert last_decrypted is not None
260 ciphertexts_are_randomized = len(set(input_hashes)) == len(input_hashes)
261 max_absolute_error = max(all_errors)
262 result: dict[str, object] = {

```

```

263 "assignment": "single-input single-output 4x4 encrypted convolution with a 3x3 kernel",
264 "library": {"name": "TenSEAL", "version": ts.__version__, "backend": "Microsoft SEAL"},
265 "runtime": {"python": sys.version.split()[0], "platform": platform.platform()},
266 "scheme": "CKKS",
267 "parameters": {
268     **asdict(parameters),
269     "coeff_mod_bit_sizes": list(parameters.coeff_mod_bit_sizes),
270     "total_coeff_modulus_bits": parameters.total_coeff_modulus_bits,
271     "global_scale": f"2^{parameters.global_scale_bits}",
272 },
273 "convolution": {
274     "input_shape": [1, 1, 4, 4],
275     "kernel_shape": [1, 1, 3, 3],
276     "output_shape": [1, 1, 2, 2],
277     "stride": 1,
278     "padding": 0,
279     "operation_semantics": "cross-correlation (CNN Conv2D convention; kernel is not flipped)",
280     "input": [[float(value) for value in row] for row in image],
281     "kernel": [[float(value) for value in row] for row in kernel],
282     "plaintext_reference": expected,
283     "decrypted_output": last_decrypted,
284 },
285 "packing": {
286     "method": "TenSEAL im2col_encoding + conv2d_im2col",
287     "valid_windows": 4,
288     "encrypted_vector_size": encoded_slot_count,
289     "logical_output_slots": 4,
290 },
291 "key_separation": {
292     "client_has_secret_key": client_context.has_secret_key(),
293     "server_has_secret_key": server_context.has_secret_key(),
294     "server_context_is_private": server_context.is_private(),
295     "server_context_is_public": server_context.is_public(),
296     "evaluation_context_excludes_secret_key": not server_context.has_secret_key(),
297 },
298 "serialized_sizes_bytes": {
299     "client_private_context_serialization": len(secret_context_bytes),
300     "server_public_evaluation_context": len(evaluation_context_bytes),
301     "encrypted_im2col_input": input_ciphertext_size,
302     "encrypted_output": output_ciphertext_size,
303     "plaintext_input_payload": 16 * 8,
304     "plaintext_output_payload": 4 * 8,
305 },
306 "verification": {
307     "trials": trials,
308     "absolute_error_tolerance": tolerance,
309     "maximum_absolute_error_all_trials": max_absolute_error,
310     "mean_absolute_error_all_slots": statistics.fmean(all_errors),
311     "all_trials_within_tolerance": all(record["within_tolerance"] for record in trial_records),
312     "ciphertexts_are_randomized": ciphertexts_are_randomized,
313     "unique_input_ciphertexts": len(set(input_hashes)),
314     "result_correct": max_absolute_error <= tolerance and ciphertexts_are_randomized,
315 },
316 "timing": {
317     "context_and_key_setup_ms": context_setup_ms,
318     "client_encrypt_and_serialize": _timing_summary(encryption_times),
319     "server_load_evaluate_and_serialize": _timing_summary(evaluation_times),
320     "client_load_and_decrypt": _timing_summary(decryption_times),
321 },
322 "trials": trial_records,

```

```

323     }
324     return result
325
326
327 def _print_summary(result: dict[str, object]) -> None:
328     convolution = result["convolution"]
329     verification = result["verification"]
330     sizes = result["serialized_sizes_bytes"]
331     timing = result["timing"]
332     key_separation = result["key_separation"]
333     print("TEN SEAL CKKS ENCRYPTED CONVOLUTION")
334     print("=====")
335     print(f"input shape   : {convolution['input_shape']}")
336     print(f"kernel shape  : {convolution['kernel_shape']}")
337     print(f"output shape   : {convolution['output_shape']}")
338     print(f"stride/padding: {convolution['stride']}/{convolution['padding']}")
339     print(f"plain result   : {convolution['plaintext_reference']}")
340     print(f"decrypt result: {convolution['decrypted_output']}")
341     print(f"max abs error : {verification['maximum_absolute_error_all_trials']:.9e}")
342     print(f"tolerance      : {verification['absolute_error_tolerance']:.1e}")
343     print(f"trials passed  : {verification['trials']}/{verification['trials']}")
344     print(f"randomized ct  : {verification['unique_input_ciphertexts']}/{verification['trials']} unique")
345     print(f"server has sk  : {key_separation['server_has_secret_key']}")
346     print(f"input ct bytes: {sizes['encrypted_im2col_input']:,}")
347     print(f"output ct bytes: {sizes['encrypted_output']:,}")
348     print(f"median encrypt: {timing['client_encrypt_and_serialize']['median_ms']:.3f} ms")
349     print(f"median server : {timing['server_load_evaluate_and_serialize']['median_ms']:.3f} ms")
350     print(f"median decrypt: {timing['client_load_and_decrypt']['median_ms']:.3f} ms")
351     print(f"RESULT        : {'PASS' if verification['result_correct'] else 'FAIL'}")
352
353
354 def main() -> None:
355     parser = argparse.ArgumentParser(description=__doc__)
356     parser.add_argument("--trials", type=int, default=10)
357     parser.add_argument("--tolerance", type=float, default=1e-3)
358     parser.add_argument("--output", type=Path, default=Path("output/experiment_results.json"))
359     args = parser.parse_args()
360
361     result = run_experiment(trials=args.trials, tolerance=args.tolerance)
362     args.output.parent.mkdir(parents=True, exist_ok=True)
363     args.output.write_text(json.dumps(result, ensure_ascii=False, indent=2), encoding="utf-8")
364     _print_summary(result)
365     print(f"evidence json : {args.output.resolve()}")
366     if not result["verification"]["result_correct"]:
367         raise SystemExit(1)
368
369
370 if __name__ == "__main__":
371     main()

```

B.2 test_fhe_convolution.py

```

1  """Regression tests for the encrypted convolution assignment."""
2
3  from __future__ import annotations
4
5  import unittest

```

```

6
7 from fhe_convolution import (
8     CKKSParameters,
9     DEFAULT_INPUT,
10    DEFAULT_KERNEL,
11    client_decrypt_output,
12    client_encrypt_im2col,
13    create_client_context,
14    load_server_context,
15    plaintext_valid_convolution,
16    serialize_evaluation_context,
17    server_encrypted_convolution,
18 )
19
20
21 class EncryptedConvolutionTests(unittest.TestCase):
22     @classmethod
23     def setUpClass(cls) -> None:
24         cls.client_context = create_client_context(CKKSParameters())
25         cls.server_context = load_server_context(serialize_evaluation_context(cls.client_context))
26
27     def test_plaintext_reference(self) -> None:
28         self.assertEqual(
29             plaintext_valid_convolution(DEFAULT_INPUT, DEFAULT_KERNEL),
30             [[26.0, 31.0], [46.0, 51.0]],
31         )
32
33     def test_shape_and_stride_validation(self) -> None:
34         with self.assertRaisesRegex(ValueError, "shape 4x4"):
35             plaintext_valid_convolution([[1.0]], DEFAULT_KERNEL)
36         with self.assertRaisesRegex(ValueError, "stride=1"):
37             plaintext_valid_convolution(DEFAULT_INPUT, DEFAULT_KERNEL, stride=2)
38
39     def test_server_has_no_secret_key(self) -> None:
40         self.assertTrue(self.client_context.is_private())
41         self.assertTrue(self.client_context.has_secret_key())
42         self.assertFalse(self.server_context.is_private())
43         self.assertFalse(self.server_context.has_secret_key())
44
45     def test_encrypted_convolution_matches_reference(self) -> None:
46         encrypted_input, windows, _ = client_encrypt_im2col(self.client_context, DEFAULT_INPUT)
47         encrypted_output = server_encrypted_convolution(
48             self.server_context,
49             encrypted_input,
50             DEFAULT_KERNEL,
51             windows,
52         )
53         actual = client_decrypt_output(self.client_context, encrypted_output, windows)
54         expected = plaintext_valid_convolution(DEFAULT_INPUT, DEFAULT_KERNEL)
55         for actual_row, expected_row in zip(actual, expected):
56             for actual_value, expected_value in zip(actual_row, expected_row):
57                 self.assertAlmostEqual(actual_value, expected_value, delta=1e-3)
58
59     def test_ckks_encryption_is_randomized(self) -> None:
60         first, _, _ = client_encrypt_im2col(self.client_context, DEFAULT_INPUT)
61         second, _, _ = client_encrypt_im2col(self.client_context, DEFAULT_INPUT)
62         self.assertNotEqual(first, second)
63
64
65 if __name__ == "__main__":

```

```
66 unittest.main(verbosity=2)
```

B.3 visualize_results.py

```
1  """Generate report figures from the encrypted-convolution evidence JSON."""
2
3  from __future__ import annotations
4
5  import json
6  from pathlib import Path
7
8  import matplotlib.pyplot as plt
9  from matplotlib.font_manager import FontProperties
10 import numpy as np
11
12
13 ROOT = Path(__file__).resolve().parent
14 RESULTS = ROOT / "output" / "experiment_results.json"
15 FIGURES = ROOT / "output" / "figures"
16 FONT_PATH = Path(r"C:\Windows\Fonts\msyh.ttc")
17 CN = FontProperties(fname=str(FONT_PATH)) if FONT_PATH.exists() else None
18 MONO = FontProperties(family="DejaVu Sans Mono")
19
20
21 def _annotated_matrix(ax, data, title: str, cmap: str, value_format: str = ".0f") -> None:
22     array = np.asarray(data, dtype=float)
23     image = ax.imshow(array, cmap=cmap, aspect="equal")
24     for row in range(array.shape[0]):
25         for col in range(array.shape[1]):
26             ax.text(col, row, format(array[row, col], value_format), ha="center", va="center", fontsize=11)
27     ax.set_title(title, fontproperties=CN, fontsize=13, pad=10)
28     ax.set_xticks(range(array.shape[1]))
29     ax.set_yticks(range(array.shape[0]))
30     ax.tick_params(length=0)
31     return image
32
33
34 def convolution_figure(result: dict) -> None:
35     convolution = result["convolution"]
36     expected = np.asarray(convolution["plaintext_reference"], dtype=float)
37     decrypted = np.asarray(convolution["decrypted_output"], dtype=float)
38     error = np.abs(decrypted - expected)
39
40     fig, axes = plt.subplots(1, 4, figsize=(13.2, 3.8), constrained_layout=True)
41     _annotated_matrix(axes[0], convolution["input"], "4x4 明文输入", "Blues")
42     _annotated_matrix(axes[1], convolution["kernel"], "3x3 明文卷积核", "RdBu_r")
43     _annotated_matrix(axes[2], decrypted, "解密后的 2x2 输出", "YlGn")
44     _annotated_matrix(axes[3], error, "逐槽绝对误差", "OrRd", ".2e")
45     fig.suptitle("CKKS 密文卷积结果与误差", fontproperties=CN, fontsize=17)
46     FIGURES.mkdir(parents=True, exist_ok=True)
47     fig.savefig(FIGURES / "encrypted_convolution_result.png", dpi=220, bbox_inches="tight")
48     plt.close(fig)
49
50
51 def runtime_figure(result: dict) -> None:
52     verification = result["verification"]
53     timing = result["timing"]
```

```

54 sizes = result["serialized_sizes_bytes"]
55 keys = result["key_separation"]
56 parameters = result["parameters"]
57 convolution = result["convolution"]
58
59 fig, ax = plt.subplots(figsize=(12.8, 7.2))
60 fig.patch.set_facecolor("#10161d")
61 ax.set_facecolor("#10161d")
62 ax.axis("off")
63 lines = [
64     ("TENSEAL CKKS ENCRYPTED CONVOLUTION", "#7ee787", 18),
65     (" $ python fhe_convolution.py --trials 10", "#b6beca", 10),
66     (" ", "white", 8),
67     (f"[OK] shapes      {convolution['input_shape']} x {convolution['kernel_shape']} -> {convolution['output_shape']}", "#7ee787", 10),
68     (f"[OK] parameters  N={parameters['poly_modulus_degree']}, Q-bits={parameters['coeff_mod_bit_sizes']}, scale={parameters['global_scale']}", "#7ee787", 10),
69     (f"[OK] key split   server_has_secret_key={keys['server_has_secret_key']}", "#7ee787", 10),
70     (f"[OK] expected    {convolution['plaintext_reference']}", "#7ee787", 10),
71     (f"[OK] decrypted   {[round(v, 6) for v in row] for row in convolution['decrypted_output']}", "#7ee787", 10),
72     (f"[OK] max error   {verification['maximum_absolute_error_all_trials']:.3e} <= {verification['absolute_error_tolerance']:.1e}", "#7ee787", 10),
73     (f"[OK] randomized {verification['unique_input_ciphertexts']}/{verification['trials']} unique ciphertexts", "#7ee787", 10),
74     (" ", "white", 8),
75     (f"[SIZE] eval context {sizes['server_public_evaluation_context']:,} B | input ct {sizes['encrypted_input']:,} B | output ct {sizes['encrypted_output']:,} B", "#8bd5ff", 9.5),
76     (f"[TIME] median    encrypt {timing['client_encrypt_and_serialize']['median_ms']:.3f} ms | server {timing['server_load_evaluate_and_serialize']['median_ms']:.3f} ms | decrypt {timing['client_load_and_decrypt']['median_ms']:.3f} ms", "#8bd5ff", 9.5),
77     (" ", "white", 8),
78     ("RESULT: ALL CORRECTNESS AND KEY-ISOLATION CHECKS PASSED", "#ffcc66", 12.5),
79 ]
80 y = 0.94
81 for text, color, size in lines:
82     ax.text(0.04, y, text, transform=ax.transAxes, color=color, fontproperties=MONO, fontsize=size, va="top")
83     y -= 0.061 if text else 0.030
84 fig.savefig(FIGURES / "runtime_validation.png", dpi=220, bbox_inches="tight", facecolor=fig.get_facecolor())
85 plt.close(fig)
86
87
88 def main() -> None:
89     result = json.loads(RESULTS.read_text(encoding="utf-8"))
90     convolution_figure(result)
91     runtime_figure(result)
92     print(f"rendered 2 figures in {FIGURES}")
93
94
95 if __name__ == "__main__":
96     main()

```

参考文献

- [1] Jung Hee Cheon, Andrey Kim, Miran Kim, Yongsoo Song. *Homomorphic Encryption for Arithmetic of Approximate Numbers*. ASIACRYPT 2017; Cryptology ePrint Archive, Paper 2016/421.
<https://eprint.iacr.org/2016/421>. 访问日期: 2026-07-24.
- [2] Ayoub Benaissa, Bilal Retiat, Bogdan Cebere, Alaa Eddine Belfedhal. *TenSEAL: A Library for Encrypted Tensor Operations Using Homomorphic Encryption*. arXiv:2104.03152, 2021.
<https://arxiv.org/abs/2104.03152>; <https://github.com/OpenMined/TenSEAL>. 访问日期: 2026-07-24.
- [3] OpenMined. *TenSEAL Tutorial 4: Encrypted Convolution on MNIST*.
<https://github.com/OpenMined/TenSEAL/blob/main/tutorials/Tutorial%20%20-%20Encrypted%20Convolution%20on%20MNIST.ipynb>. 访问日期: 2026-07-24.
- [4] Microsoft Research. *Microsoft SEAL: An Easy-to-use Open-source Homomorphic Encryption Library*.
<https://github.com/microsoft/SEAL>. 访问日期: 2026-07-24.
- [5] HomomorphicEncryption.org. *Homomorphic Encryption Standard, Version 1.1*. 2018.
<https://homomorphicencryption.org/standard/>. 访问日期: 2026-07-24.
- [6] Vincent Dumoulin, Francesco Visin. *A Guide to Convolution Arithmetic for Deep Learning*. arXiv:1603.07285, 2016.
<https://arxiv.org/abs/1603.07285>. 访问日期: 2026-07-24.
- [7] 课程参考资料: 全同态密码算法和应用, 本地 HTML 课件, 2026.
- [8] 付勇. 对称密码的软件实现与优化. 课程参考资料, 2026.
- [9] 课程参考资料: SM3 软件实现与优化. 2026.